I. THE CONCEPT: A GAME OF CAT AND MOUSE

Before GANs, machines were mostly used for *discriminative* tasks—taking an input and predicting a label (e.g., "Is this a cat?"). In 2014, Ian Goodfellow introduced **GANs**, which shifted the focus to *generative* tasks—creating new, synthetic data that is indistinguishable from real data.

The core idea is based on Game Theory: two neural networks compete against each other in a zero-sum game.

1.1. The Two Players

1. **The Generator (G):** An artist that attempts to create realistic fake data (e.g., images). It starts with random noise and tries to transform it into an image that looks like a real photo.
2. **The Discriminator (D):** A critic that inspects images and decides whether they are "Real" (from the dataset) or "Fake" (from the Generator).

II. THE TRAINING DYNAMICS

The training process is a continuous loop where both players improve:

- **The Generator** tries to maximize the probability of the Discriminator making a mistake.
- **The Discriminator** tries to minimize its own error rate in distinguishing real vs. fake.

Eventually, if the training is balanced, the **Generator** becomes so good that the **Discriminator** can no longer tell the difference between real and synthetic data. This is called the **Nash Equilibrium**.

III. MATHEMATICAL FRAMEWORK

GANs use a **Minimax Loss Function**, where D tries to maximize the accuracy of its predictions, and G tries to minimize it.

$$\min_G \max_D V(D,G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

- $\mathbb{E}_{x \sim p_{data}}$: Expectation over real data.
- $\mathbb{E}_{z \sim p_z(z)}$: Expectation over random noise (z).

IV. CHALLENGES IN TRAINING

GANs are notoriously difficult to train compared to standard CNNs.

1. **Mode Collapse:** The Generator discovers one specific image that "fools" the Discriminator (e.g., a perfect photo of a dog) and keeps producing only that one image, ignoring the diversity of the

training data.
2. **Vanishing Gradients:** If the Discriminator is too strong (too perfect), the Generator receives no useful feedback on how to improve.
3. **Non-Convergence:** Because it is a game, the two networks can oscillate, constantly chasing each other without ever finding an equilibrium.

V. ARCHITECTURAL EVOLUTION

- **DCGAN (Deep Convolutional GAN):** Used convolutional layers for stable image generation, solving many early training issues.
- **CycleGAN:** Capable of "Style Transfer" (e.g., turning a photo of a horse into a zebra without needing paired images).
- **StyleGAN:** The gold standard for high-resolution human face synthesis. It introduces a "mapping network" to control specific attributes like hair, pose, and age.

VI. PRACTICAL LAB: BUILDING A SIMPLE GAN (PYTORCH)

Python

```
import torch

import torch.nn as nn


# The Generator: Transforms random noise (z) into an image

class Generator(nn.Module):

    def __init__(self, latent_dim, img_shape):

        super().__init__()

        self.model = nn.Sequential(

            nn.Linear(latent_dim, 128),

            nn.ReLU(),

            nn.Linear(128, img_shape),

            nn.Tanh() # Tanh maps outputs to [-1, 1]

        )

    def forward(self, z): return self.model(z)


# The Discriminator: Binary Classifier (Real vs Fake)

class Discriminator(nn.Module):
```

```
def __init__(self, img_shape):
    super().__init__()
    self.model = nn.Sequential(
        nn.Linear(img_shape, 128),
        nn.LeakyReLU(0.2),
        nn.Linear(128, 1),
        nn.Sigmoid() # Output probability (0 to 1)
    )
def forward(self, img): return self.model(img)
```